

Documentação API de Endereços

Nome: Jackeline de Paula Lima

Turma: 6º semestre de SI

Matéria: Arquitetura Orientada a Serviços (SOA)

1. Introdução

Esta atividade tem como objetivo documentar e implementar uma API de endereços para o projeto loja-virtual. A API permite realizar operações de cadastro, consulta, atualização e remoção de endereços.

Código

2. Model

A **model Endereco** representa a entidade de endereço no sistema. Ela contém os campos necessários baseados na tabela fornecida, onde também foi adicionada uma relação com cliente, permitindo que um cliente possua múltiplos endereços.

```
@Entity
@Table(name = "Enderecos")
@Data
public class Endereco extends ModeloBaseAuditavel{
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    /* ATIVIDADE PARA ENTREGAR */
    private String logradouro;

    private String bairro;

    private Integer numero; //integer é igual ao int mas permite valor nulo

    private String complemento;

    @Column(length = 9)
    private String cep;

    private String cidade;

    private String estado;

    //conectando com o cliente
    @ManyToOne
    @JoinColumn(name = "cliente_id", nullable = true)
    private Cliente cliente;
}
```

*** Foi utilizado Integer pois há endereços que não possuem número (principalmente casas/chácaras de interior)*

3. Repository

O **repository** é responsável pelo acesso ao banco de dados, ele utiliza JpaRepository, permitindo operações como:

- salvar
- buscar por id
- listar
- deletar

Sem necessidade de implementação manual dessas operações.

```
package br.org.sp.fatec.lojavirtual.repository;

import br.org.sp.fatec.lojavirtual.model.Endereco;
import org.springframework.data.jpa.repository.JpaRepository;

public interface EnderecoRepository extends JpaRepository<Endereco, Long> {

}
```

4. DTOs

Os DTOs são utilizados para controlar os dados enviados e recebidos pela API.

- DTO de entrada: valida os dados recebidos

```
@Data 6 usages
public class EnderecoFormDto {

    @NotBlank
    private String logradouro;

    @NotBlank
    private String bairro;

    private Integer numero;

    private String complemento;

    @NotBlank
    @Pattern(regexp = "\\d{5}-?\\d{3}$", message = "CEP
    deve estar no formato 00000-000")
    private String cep;

    @NotBlank
    private String cidade;

    @NotBlank
    private String estado;

    @NotNull(message = "Id do cliente deve ser informado")
    private Long clienteId;
}
```

**Deixando especificado os campos obrigatórios e validação do cep*

- DTO de saída: define os dados retornados ao usuário

```
@Data 14 usages
public class EnderecoResultDto {

    private Long id;
    private String logradouro;
    private String bairro;
    private Integer numero;
    private String complemento;
    private String cep;
    private String cidade;
    private String estado;
    private Long clienteId;
}
```

5. Wrapper

O wrapper é utilizado para converter entidades em DTOs. Permite reutilização da lógica de conversão em diferentes partes do sistema.

```
public class EnderecoWrapper { 5 usages

    public static EnderecoResultDto converterEnderecoDto
        (Endereco endereco) { 4 usages

        EnderecoResultDto dto = new EnderecoResultDto();

        dto.setId(endereco.getId());
        dto.setLogradouro(endereco.getLogradouro());
        dto.setBairro(endereco.getBairro());
        dto.setNumero(endereco.getNumero());
        dto.setComplemento(endereco.getComplemento());
        dto.setCep(endereco.getCep());
        dto.setCidade(endereco.getCidade());
        dto.setEstado(endereco.getEstado());

        if (endereco.getClientes() != null) {
            dto.setClienteId(endereco.getClientes().getId());
        }

        return dto;
    }
}
```

6. Service

O service é responsável pelas regras de negócio e pela comunicação com o repository.

Principais funções:

- salvar endereço
- listar endereços
- buscar por id
- deletar endereço
- atualizar endereço

```
@Service
public class EnderecoService {

    private final EnderecoRepository
        enderecoRepository; 6 usages
    private final ClienteRepository clienteRepository; 3 usages

    public EnderecoService(EnderecoRepository
        enderecoRepository, no usages
                           ClienteRepository clienteRepository) {
        this.enderecoRepository = enderecoRepository;
        this.clienteRepository = clienteRepository;
    }

    private Endereco falhaEnderecoNaoEncontrado(Long id)
    { 3 usages
        return enderecoRepository.findById(id)
            .orElseThrow(() -> new ResponseStatusException(
                HttpStatus.NOT_FOUND, "Endereço não
                                     encontrado"
            ));
    }
}
```

```

public EnderecoResultDto salvarEndereco (EnderecoFormDto
dto){ 1 usage
    // procura se existe o cliente para dar certeza que da para
    conectar o endereço com o cliente (deve existir cliente para
    registrar o endereço)
    Cliente cliente = clienteRepository.findById(dto.getClientId
    ())
        .orElseThrow(() -> new ResponseStatusException(
            HttpStatus.NOT_FOUND, "Cliente não encontrado"
        ));

    // registra os dados do endereço
    Endereco enderecoNovo = new Endereco();
    enderecoNovo.setLogradouro(dto.getLogradouro());
    enderecoNovo.setBairro(dto.getBairro());
    enderecoNovo.setNumero(dto.getNumero());
    enderecoNovo.setCep(dto.getCep());
    enderecoNovo.setCidade(dto.getCidade());
    enderecoNovo.setEstado(dto.getEstado());
    enderecoNovo.setCliente(cliente);

    // salva
    enderecoNovo = enderecoRepository.save(enderecoNovo);

    //retorno do endereço registrado
    return EnderecoWrapper.converterEnderecoDto(enderecoNovo);
}

```

```

public EnderecoResultDto buscarEnderecoPorId(Long id)
{ 1 usage
    Endereco endereco = falhaEnderecoNaoEncontrado(id);

    return EnderecoWrapper.converterEnderecoDto(endereco);
}

public Page<EnderecoResultDto> listarEnderecoPaginado(Pageable
paginação) { 1 usage
    return enderecoRepository.findAll(paginação)
        .map(EnderecoWrapper::converterEnderecoDto);
}

public void deletarEndereco(Long id) { 1 usage
    Endereco endereco = falhaEnderecoNaoEncontrado(id);

    enderecoRepository.delete(endereco);
}

```

```

public EnderecoResultDto atualizarEndereco(Long id,
EnderecoFormDto dto) { 1 usage

    Endereco endereco = falhaEnderecoNaoEncontrado(id);

    Cliente cliente = clienteRepository.findById(dto.getClientId()
    ())
        .orElseThrow(() -> new RuntimeException(
            HttpStatus.NOT_FOUND, "Cliente não encontrado"
        ));

    endereco.setLogradouro(dto.getLogradouro());
    endereco.setBairro(dto.getBairro());
    endereco.setNumero(dto.getNumero());
    endereco.setComplemento(dto.getComplemento());
    endereco.setCep(dto.getCep());
    endereco.setCidade(dto.getCidade());
    endereco.setEstado(dto.getEstado());
    endereco.setCliente(cliente);

    endereco = enderecoRepository.save(endereco);

    return EnderecoWrapper.converterEnderecoDto(endereco);
}

```

7. Controller

O controller expõe os endpoints da API, recebendo as requisições HTTP, chamando o service e retornando as respostas ao cliente.

```

@RestController
@RequestMapping("/enderecos")
public class EnderecoController {
    private final EnderecoService service; 6 usages

    public EnderecoController(EnderecoService service) {
        this.service = service;
    }

    @PostMapping
    public EnderecoResultDto salvar(@Valid @RequestBody
    EnderecoFormDto dto) { return service.salvarEndereco(dto);
    }

    @GetMapping("/{id}")
    public EnderecoResultDto buscar(@PathVariable Long id) {
        return service.buscarEnderecoPorId(id);
    }

    @PutMapping("/{id}")
    public EnderecoResultDto atualizar(@PathVariable Long id,
    @RequestBody @Valid EnderecoFormDto dto
    ) {
        return service.atualizarEndereco(id, dto);
    }

    @DeleteMapping("/{id}")
    public void deletar(@PathVariable Long id) { service
        .deletarEndereco(id);
    }

    @GetMapping
    public Page<EnderecoResultDto> listar(Pageable paginacao)
    { return service.listarEnderecoPaginado(paginacao);
    }
}

```

8. Documentação da API

- **Criar endereço**

Requisição:

POST /enderecos

Corpo:

```
{
  "clienteld": number,
  "logradouro": string,
  "bairro": string,
  "numero": number,
  "complemento": string,
  "cep": string,
  "cidade": string,
  "estado": string
}
```

Validações:

- logradouro, bairro, cep, cidade, estado e id do cliente são itens obrigatórios.
- cep é validado no formato 00000-000

Resposta:

- 201 - Criado
- 400 - Dados inválidos

Corpo resposta:

```
{
  "clienteld": number,
  "logradouro": string,
  "bairro": string,
  "numero": number,
  "complemento": string,
  "cep": string,
  "id": number,
  "cidade": string,
  "estado": string
}
```

-
- **Buscar endereço por ID**

Requisição:

GET /enderecos/{id}

Resposta:

- 200 - Registro encontrado
- 404 - Registro não encontrado

Corpo

```
{  
  
}
```

Corpo resposta:

```
{  
  "id": number,  
  "logradouro": string,  
  "bairro": string,  
  "numero": number,  
  "complemento": string,  
  "cep": string,  
  "cidade": string,  
  "estado": string  
}
```

- **Listar endereços (paginado)**

Requisição:

GET http://localhost:8080/enderecos?page=0&size=2

Parâmetros:

- page: número da página
- size: quantidade de registros

Resposta:

- 200 - Registro encontrado
- 404 - Registros não encontrados

Corpo resposta:

```
{
  "content": [
    {
      "bairro": "Centro",
      "cep": "01000-000",
      "cidade": "São Paulo",
      "clienteld": 1,
      "complemento": null,
      "estado": "SP",
      "id": 1,
      "logradouro": "Av. Principal",
      "numero": 100
    }
  ],
  "empty": false,
  "first": true,
  "last": true,
  "number": 0,
  "numberOfElements": 1,
  "pageable": {
    "offset": 0,
    "pageNumber": 0,
    "pageSize": 2,
    "paged": true,
    "sort": {
      "empty": true,
      "sorted": false,
      "unsorted": true
    },
    "unpaged": false
  },
  "size": 2,
  "sort": {
    "empty": true,
    "sorted": false,
    "unsorted": true
  },
  "totalElements": 1,
  "totalPages": 1
}
```

- **Atualizar endereço**

Requisição:

PUT /enderecos/{id}

Corpo:

```
{
  "clienteld": number,
  "logradouro": string,
  "bairro": string,
  "numero": number,
  "complemento": string,
  "cep": string,
  "cidade": string,
  "estado": string
}
```

Resposta:

- 200 - Atualizou com sucesso
- 404 - Registro não encontrado

Corpo resposta:

```
{
  "clienteld": number,
  "logradouro": string,
  "bairro": string,
  "numero": number,
  "complemento": string,
  "cep": string,
  "cidade": string,
  "estado": string
}
```

- **Deletar endereço**

Requisição:

DELETE /enderecos/{id}

Resposta:

- 204 - Removido com sucesso
 - 404 - Registro não encontrado
-

9. Considerações Finais

A API de endereços foi estruturada seguindo boas práticas de arquitetura em camadas, utilizando DTOs para controle de dados e separação de responsabilidades entre controller, service e repository.

A documentação apresentada garante clareza no uso da API e facilita sua integração com outros sistemas.